

UNIVERSIDADE CATÓLICA DE BRASÍLIA

Curso de Sistemas de Informação | Disciplina de Banco de Dados

SISTEMA DE GESTÃO PARA CLÍNICA ODONTOLÓGICA

Artigo Técnico — Disciplina de Banco de Dados

Antônio Joaquim Alves
Cayo do Prado Santos
Dankley Meireles de Lima
Felipe de Sousa Lopes

Brasília – DF · 2026

RESUMO

Este artigo descreve o desenvolvimento de um sistema de gestão para clínica odontológica utilizando banco de dados relacional MySQL e banco de dados NoSQL MongoDB Atlas, com integração entre backend em Node.js (Express) e frontend em HTML, CSS e JavaScript. O sistema contempla o gerenciamento de consultas, pacientes, dentistas, estoque de insumos, movimentações financeiras e prontuários clínicos. A modelagem do banco de dados seguiu as etapas conceitual, lógica e física, resultando em dez tabelas relacionadas com uso de triggers, views, stored procedures, functions, índices e controle de acesso por usuários. O MongoDB Atlas foi adotado para armazenamento de prontuários em formato de documento, dados de natureza não estruturada que variam por paciente. A aplicação demonstra o domínio dos principais recursos de um SGBD relacional e NoSQL em um contexto real e relevante para o Distrito Federal.

Palavras-chave: banco de dados relacional; MySQL; MongoDB; clínica odontológica; Node.js; gestão de sistemas.

1. INTRODUÇÃO

O Distrito Federal apresenta um dos maiores índices de dentistas por habitante do Brasil. Segundo dados do Conselho Federal de Odontologia (CFO), há aproximadamente 9.343 cirurgiões-dentistas registrados no CRO-DF, resultando em uma proporção de 1 profissional para cada 326 habitantes — quatro vezes acima da recomendação da Organização Mundial da Saúde (OMS), que sugere 1 para cada 1.200 habitantes.

Diante desse cenário, muitos dentistas, especialmente os recém-formados, registram consultas e pacientes em papéis avulsos ou utilizam softwares dispendiosos e incompatíveis entre si. Com o objetivo de democratizar o acesso à tecnologia de gestão clínica, este trabalho propõe o desenvolvimento de um sistema integrado de gerenciamento para clínicas odontológicas, fundamentado em banco de dados relacional MySQL, banco de dados NoSQL MongoDB Atlas e acessível via interface web.

O sistema permite o cadastro e controle de pacientes, agendamento de consultas, gestão de estoque de insumos, controle financeiro, emissão de relatórios e registro de prontuários clínicos, promovendo eficiência operacional e segurança no armazenamento de dados.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um sistema de gestão para clínica odontológica com banco de dados relacional MySQL e banco NoSQL MongoDB Atlas, integrando frontend e backend, demonstrando o domínio dos principais recursos de um SGBD.

2.2 Objetivos Específicos

- Modelar o banco de dados nas etapas conceitual, lógica e física, aplicando normalização e integridade referencial;
- Implementar triggers, views, stored procedures, functions e índices devidamente justificados;
- Definir controle de acesso com usuários distintos, sem uso do root;
- Criar IDs customizados para dados críticos, como consultas, utilizando function específica;
- Desenvolver API REST em Node.js/Express que se comunica com MySQL e MongoDB Atlas;
- Armazenar prontuários clínicos no MongoDB Atlas, justificando o uso do NoSQL para dados não estruturados;
- Construir interface web que exiba dados das views MySQL e permita registro de prontuários via MongoDB.

3. METODOLOGIA

O desenvolvimento do sistema seguiu o ciclo de vida clássico de projetos de banco de dados: levantamento de requisitos, modelagem conceitual, modelagem lógica, modelagem física, implementação, integração e testes.

Na fase de levantamento, foram identificados os processos essenciais de uma clínica odontológica: cadastro de usuários (dentistas e pacientes), agendamento de consultas, controle de estoque, gestão financeira e registro de prontuários clínicos. A partir daí, definiu-se o Modelo Entidade-Relacionamento (MER), que evoluiu para o modelo lógico relacional e, finalmente, para o script DDL em MySQL.

Para o backend, utilizou-se Node.js com o framework Express, expondo endpoints REST que consomem o banco de dados via drivers mysql2 e mongodb. O frontend foi desenvolvido com HTML5, CSS3 e JavaScript puro, garantindo leveza e compatibilidade. A comunicação entre as camadas segue o padrão cliente-servidor, com a interface consultando as views do MySQL para dados estruturados e o MongoDB Atlas para prontuários.

4. DESCRIÇÃO DO SISTEMA

4.1 Funcionalidades

- Agenda completa exibindo dados da view vw_agenda_completa com cards de resumo;
- Relatório de faturamento por dentista via view vw_faturamento_dentistas;
- Controle de estoque vencido via view vw_estoque_vencido;
- Registro e consulta de prontuários clínicos armazenados no MongoDB Atlas;
- Agendamento de nova consulta via stored procedure sp_agendar_consulta;
- Controle financeiro com status de pagamento (receitas e despesas).

4.2 Tecnologias Utilizadas

A tabela a seguir apresenta as tecnologias utilizadas em cada camada do sistema com suas respectivas justificativas.

Camada	Tecnologia	Justificativa
Banco Relacional	MySQL 8.0	SGBD relacional robusto com suporte completo a triggers, procedures, views e controle de acesso
Banco NoSQL	MongoDB Atlas	Banco de documentos para prontuários clínicos — dados não estruturados sem schema fixo
Backend	Node.js + Express	Runtime JavaScript no servidor; Express leve para APIs REST; drivers mysql2 e mongodb nativos
Conector MySQL	mysql2	Driver Node.js para MySQL com suporte a Promises e queries parametrizadas
Conector MongoDB	mongodb (oficial)	Driver oficial MongoDB para Node.js com suporte a Atlas e operações assíncronas
Frontend	HTML5 + CSS3 + JS	Tecnologias padrão da web sem frameworks adicionais para reduzir complexidade

5. MODELAGEM DO BANCO DE DADOS

5.1 Diagrama Entidade-Relacionamento (DER)

O Diagrama Entidade-Relacionamento representa a estrutura conceitual do banco de dados, evidenciando as entidades, seus atributos e os relacionamentos entre elas. A entidade usuarios é a base do sistema, da qual herdam as especializações cliente e dentista — padrão conhecido como herança por delegação.

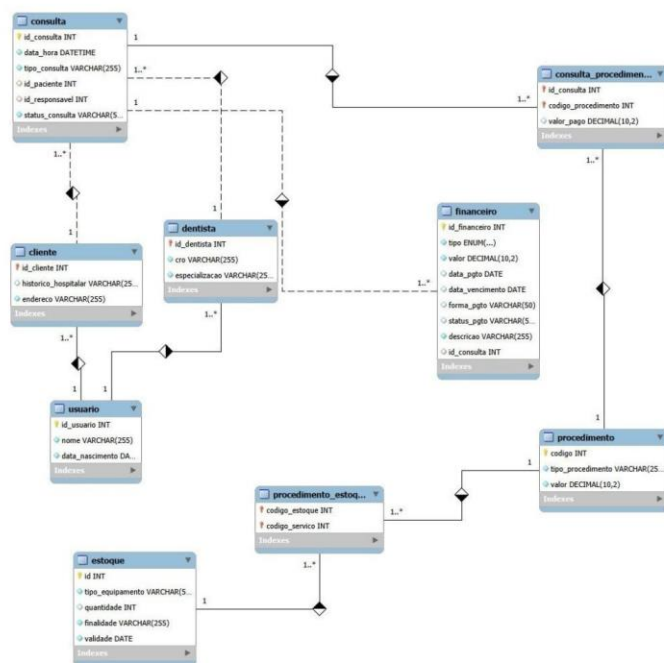


Figura 1 — Diagrama Entidade-Relacionamento (MER) do banco clinica_odontologica

5.2 Entidades e Relacionamentos

O banco de dados clinica_odontologica é composto por dez tabelas que modelam os processos clínicos, administrativos e financeiros da clínica.

Tabela	Descrição
grupos_usuarios	Agrupa usuários por perfil de acesso (Administrador, Dentista, Recepcionista). Tabela obrigatória para controle de permissões.
usuarios	Entidade-base de todos os usuários do sistema (login, senha, nome, grupo). Tabela obrigatória para autenticação e rastreabilidade.
cliente	Especialização de usuarios com histórico hospitalar e endereço do paciente.
dentista	Especialização de usuarios com CRO e especialização clínica do profissional.
consulta	Agendamentos com data, tipo, status, paciente e dentista responsável. IDs gerados por function customizada.
procedimento	Catálogo de procedimentos odontológicos com valor.
consulta_procedimento	Tabela associativa N:N entre consulta e procedimento.
financeiro	Registros de receitas e despesas vinculados ou não a consultas.
estoque	Controle de insumos com quantidade e validade.
procedimento_estoque	Associação N:N entre procedimento e estoque (consumo de insumos por procedimento).

5.3 Índices

Índice	Tabela / Coluna	Justificativa
idx_consulta_data	consulta(data_hora)	Consultas de agenda filtram massivamente por data/hora. O índice reduz o custo de $O(N)$ para $O(\log N)$, essencial em operações frequentes de leitura.
idx_financeiro_status	financeiro(status_pgto)	Relatórios financeiros filtram constantemente por status de pagamento. O índice acelera JOINS e cláusulas WHERE sobre essa coluna.

5.4 Triggers

Trigger	Evento	Função
tr_baixa_estoque_automatica	AFTER INSERT em consulta_procedimento	Desconta automaticamente os insumos utilizados da tabela estoque ao vincular um procedimento a uma consulta, garantindo consistência sem intervenção manual.
tr_impedir_alteracao_cancelada	BEFORE UPDATE em consulta	Impede qualquer alteração em consultas com status "Cancelada" via SIGNAL SQLSTATE. Protege a integridade do histórico clínico e auditabilidade.

5.5 Views

Três views foram criadas para simplificar consultas complexas, servindo como interface de leitura consolidada para o frontend.

View	Descrição	Uso no Sistema
vw_agenda_completa	JOINS entre consulta, usuarios e financeiro, exibindo dados consolidados da agenda.	Exibida na tela de Agenda do frontend; elimina múltiplas queries na camada de aplicação.
vw_faturamento_dentistas	Agrega total de atendimentos e receita por dentista, filtrando apenas pagamentos confirmados.	Alimenta o relatório de faturamento; concentra lógica de negócio no banco.
vw_estoque_vencido	Filtra itens com validade anterior à data atual (CURDATE()).	Exibida na tela de Estoque Vencido; readonly_dashboard acessa apenas esta view, nunca a tabela diretamente.

A figura abaixo exhibe a view vw_agenda_completa sendo consultada diretamente no MySQL Workbench, confirmando a exibição correta de dados consolidados.



id_usuario	nome	data_nascimento
1	Cayo Santos	2000-05-15
2	Antônio Joaquim	1985-10-20
3	Ana Oliveira	1992-03-08
4	Ricardo Silva	1978-12-12
5	Mariana Costa	1995-07-25
6	Lucas Pereira	1988-01-30

Figura 2 — Resultado da consulta `SELECT * FROM vw_agenda_completa` no MySQL Workbench

5.6 Stored Procedures e Functions

Objeto	Tipo	Descrição e Justificativa
fn gerar_id_consulta()	Function	Gera ID customizado no formato "CON{ANO}-{SEQUENCIAL}" (ex.: CON2026-00001). O formato semântico permite identificar o ano da consulta pelo ID, impossível com AUTO_INCREMENT simples.
sp_agendar_consulta(...)	Procedure	Encapsula a lógica de agendamento: chama fn gerar_id_consulta() e insere o registro na tabela consulta. Garante que o ID seja sempre gerado de forma padronizada.
sp_criar_usuario(...)	Procedure	Centraliza a criação de usuários com validação básica, facilitando futuras expansões como hash de senha com bcrypt.

5.7 Usuários e Controle de Acesso

O acesso ao banco de dados é proibido via root. Foram criados dois usuários com níveis de acesso distintos, seguindo o princípio do menor privilégio.

Usuário	Permissões	Justificativa
app_odontoped_user@localhost	SELECT, INSERT, UPDATE, DELETE no schema completo	Utilizado pelo backend para todas as operações CRUD. Não possui permissão de DROP, CREATE ou ALTER, impedindo alterações estruturais acidentais.
readonly_dashboard@localhost	SELECT exclusivo nas views vw_agenda_completa, vw_faturamento_dentistas e vw_estoque_vencido	Utilizado pelo painel web. Acesso somente leitura restrito às views, sem possibilidade de leitura de tabelas brutas ou modificação de dados.

Os grupos de usuários da aplicação (tabela grupos_usuarios) definem três perfis de acesso: Administrador (acesso total ao sistema), Dentista (acesso a prontuários e agenda) e Recepcionista (acesso a cadastros e agendamentos).

5.8 Banco de Dados NoSQL — MongoDB Atlas

Além do modelo relacional, o sistema utiliza o MongoDB Atlas para persistência dos prontuários clínicos. A escolha do NoSQL para esse módulo é justificada pela natureza dos dados: anotações clínicas são textos livres que variam em estrutura e extensão de paciente para paciente, sem schema fixo e previsível.

A coleção prontuarios armazena documentos com os campos nome_paciente, dentista, anotacao e data_registro. A ausência de schema fixo permite que campos adicionais sejam incluídos no futuro sem necessidade de migração de schema — vantagem impossível no modelo relacional.

Critério	MySQL (Relacional)	MongoDB Atlas (NoSQL)
Tipo de dado	Estruturado, schema fixo	Não estruturado, schema flexível
Uso no sistema	Consultas, financeiro, estoque, usuários	Prontuários clínicos
Integridade	Garantida por FK e constraints	Responsabilidade da aplicação
Escalabilidade	Vertical (scale-up)	Horizontal (scale-out)
Justificativa	Dados com relacionamentos e regras rígidas	Dados clínicos variáveis sem estrutura previsível

6. EVIDÊNCIAS DE EXECUÇÃO

A figura abaixo apresenta o log de execução completo do script SQL no MySQL Workbench, confirmando que todas as instruções DDL e DML foram executadas sem erros.

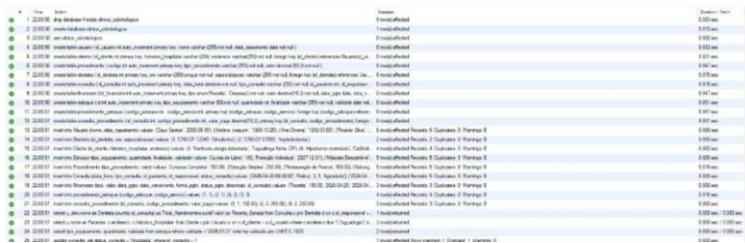


Figura 3 — Log de execução do script_clinica.sql no MySQL Workbench (26 instruções executadas com sucesso)

O log evidencia a execução das operações de criação de banco de dados, tabelas, inserção de dados, criação de procedures/triggers, execução das queries de exemplo e atualização de registros — cobrindo todo o ciclo de vida das operações especificadas.

As figuras a seguir apresentam o painel web em funcionamento, demonstrando a integração entre frontend, API Node.js, banco MySQL e MongoDB Atlas.

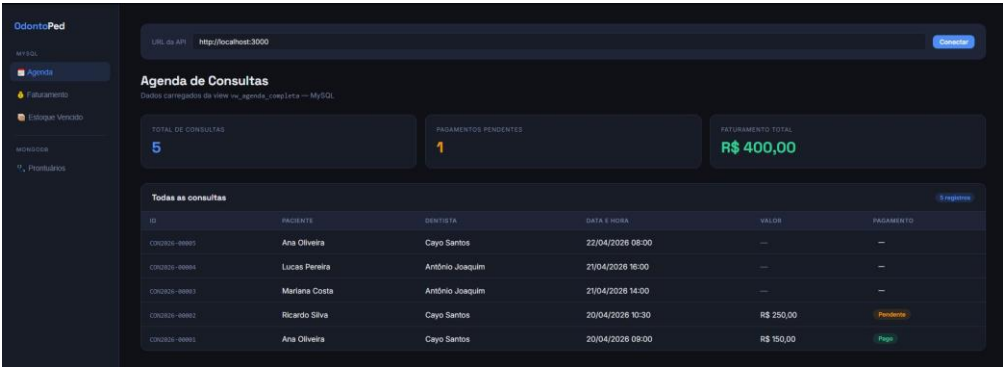


Figura 4 — Tela de Agenda carregando dados da view vw_agenda_completa via API

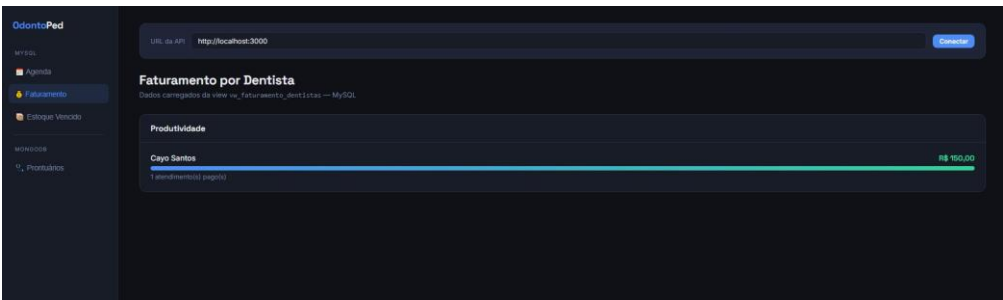


Figura 5 — Tela de Faturamento exibindo dados da view vw_faturamento_dentistas

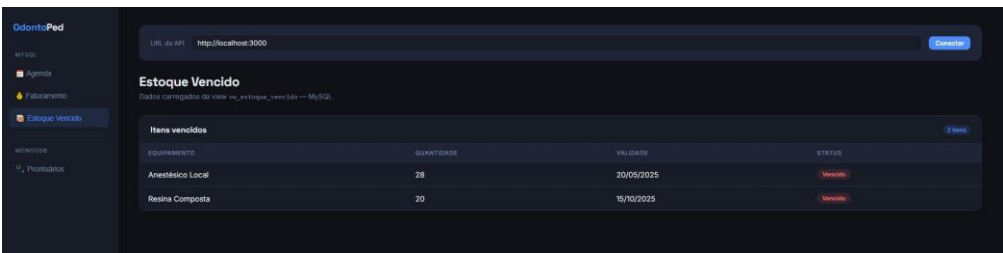


Figura 6 — Tela de Estoque Vencido exibindo dados da view vw_estoque_vencido

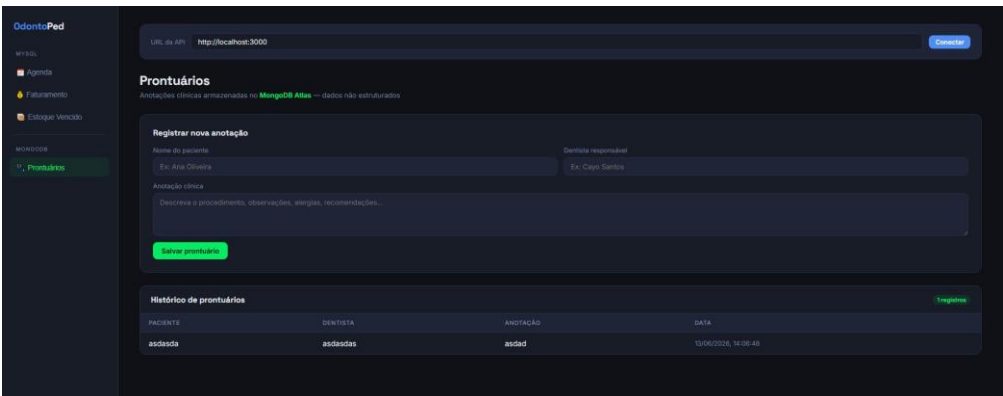


Figura 7 — Tela de Prontuários com registro e listagem de documentos no MongoDB Atlas

7. CONCLUSÃO

O sistema de gestão para clínica odontológica foi desenvolvido com sucesso, contemplando todos os requisitos técnicos estabelecidos: estrutura relacional completa com dez tabelas (incluindo as obrigatórias usuarios e grupos_usuarios), triggers de automatização e proteção de dados, três views para abstração de consultas complexas, stored procedures e functions para encapsulamento de lógica de negócio — incluindo geração customizada de IDs para dados críticos —, índices justificados para performance, e controle de acesso com dois usuários distintos sem uso do root.

A integração entre o banco de dados relacional MySQL, o banco NoSQL MongoDB Atlas, a API REST em Node.js/Express e o frontend em HTML/CSS/JS demonstrou a viabilidade de construir soluções completas com tecnologias abertas e acessíveis. A adoção de dois paradigmas de banco de dados — relacional para dados estruturados e de documentos para prontuários — reflete uma decisão arquitetural fundamentada nas características de cada tipo de dado.

Como trabalhos futuros, sugere-se a implementação de autenticação JWT na API, hash de senhas com bcrypt, paginação dos resultados nos endpoints, e possível migração do frontend para um framework como React ou Vue.js para maior escalabilidade.

8. REFERÊNCIAS

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2019.

DATE, C. J. Introdução a sistemas de bancos de dados. 8. ed. Rio de Janeiro: Campus, 2004.

MySQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: mai. 2026.

MongoDB. MongoDB Documentation. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: jun. 2026.

Express.js. Express Documentation. Disponível em: <https://expressjs.com/>. Acesso em: jun. 2026.

CONSELHO FEDERAL DE ODONTOLOGIA. Dados estatísticos. Disponível em: <https://cfo.org.br/>. Acesso em: mai. 2026.